

Architectural Decisions

- Definition
 - Mozaic, the system, and Mozaic system refers to the software system that this project is going to develop, launch, and operate.
- Goal
 - This document describes architectural decisions that implement the system requirements.
 - This document serves as additional technical terminology of the project.

1. Overall state machine

1.1 Definition

- **Asset state** is the state of what amounts of what tokens, including LP tokens, on what chains are, at the given moment of time,
 - deposited and pending staking
 - staked
 - rewarded and pending harvesting
 - staying in system treasuries
 - pending withdrawal
- **Request state** is the state of what deposit/withdrawal requests are accepted by the system and waiting for final processing, at the given moment of time.
- If the system does **Optimize asset/request state**, it changes the states to earn optimal staking reward.

1.2 Design decisions

We choose the Sleeping-then_Optimizing model for the overall system behavior.

- The system will not always be optimizing, but sleeping most of the time accepting deposit/withdrawal requests from users.
- If the system **accepts** a deposit request, it
 - collects the asset the user wants to deposit, (on the asset's chain)
 - tells the user to wait until the next optimization round, when the system will send LP tokens to the user in return for the asset,
 - and book the request with the system for later processing.
- If the system **accepts** a withdraw request, it
 - collects the LP tokens the user wants to return, (on the LP's chain)
 - tells the user to wait until the next optimization round, when the system will send some asset of the requested token type,
 - and book the request with the system for later processing.
- The system will **optimize asset/request state** at regular or irregular intervals. The frequency of optimization rounds will be optimized, as frequent moves of asset may incur more costs while infrequent optimization rounds will hinder from quick

maneuver of staking.

- When entering the **Optimization process**, or the **Optimizing** state, the system will take a snapshot of the asset/request state. Then the system transforms/changes the states to optimal states for maximum reward, while continuing to **accept** deposit/withdrawal requests, which will be handled at the next round of optimization.

1.3 Visual description

The Sleeping-then_Optimizing model of behavior can be expressed in a UML State Machine diagram shown below:

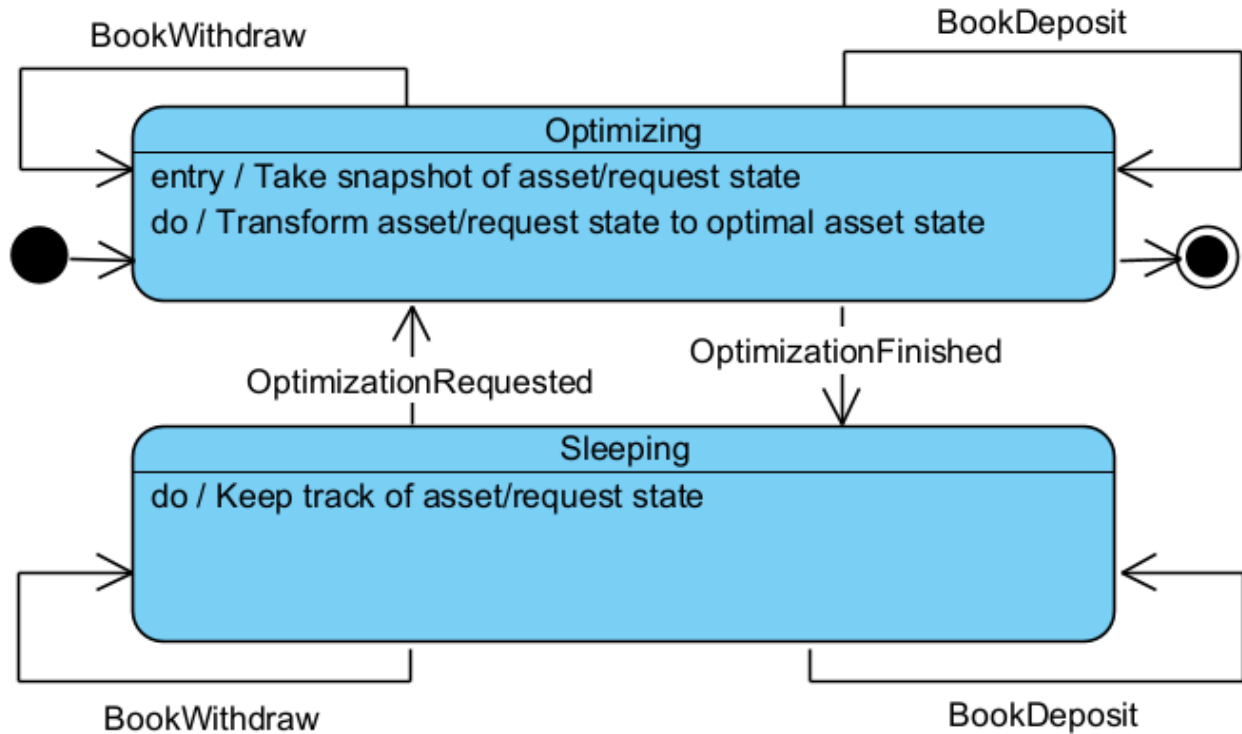


Figure. State Machine of Mozaic asset management

2. Vaults

We need a module that implements the use case **Control asset move** identified in the requirements specification, solely, and completely. We call the module the vault, for the following reasons.

2.1 Vaults as smart contract

According to the requirements, vaults are responsible to, exclusively and at the decentralization level,

- keep track of all changes to asset/request state, defined in the previous section
- execute all changes to asset/request state,
- log all changes to asset/request state.

The only way is to deploy smart contracts on chains that cooperate with each other to form a vault module. We call them vaults or vault contracts.

2.2 Vaults with limited responsibility

According to the requirements, vaults do *not* have to

- find an optimal asset state to **Optimize asset/request state** to
- execute asset move requests, like `assetMovePlan` identified in requirements, strictly, because there will not be negative profit.

2.3 Vaults' local responsibility - local transportation of token

- Pull asset from the user if a deposit involves a local token of asset to collect
- Push LP to the user if a deposit involves the local LP token to return
- Pull LP tokens from the user if a withdrawal involves the local LP token to collect
- Push asset to the user if a withdrawal involves a local token of asset to return

2.4 Vaults' local responsibility - local moves of Staking stock

- Collect rewards from local staking pools
-
- Collect rewards from local staking pools
- Pull asset from the user if a deposit involves a local token of asset to collect
- Push LP to the user if a deposit involves the local LP token to return
- Pull LP tokens from the user if a withdrawal involves the local LP token to collect
- Push asset to the user if a withdrawal involves a local token of asset to return

The design decisions are as illustrated in the following figure:

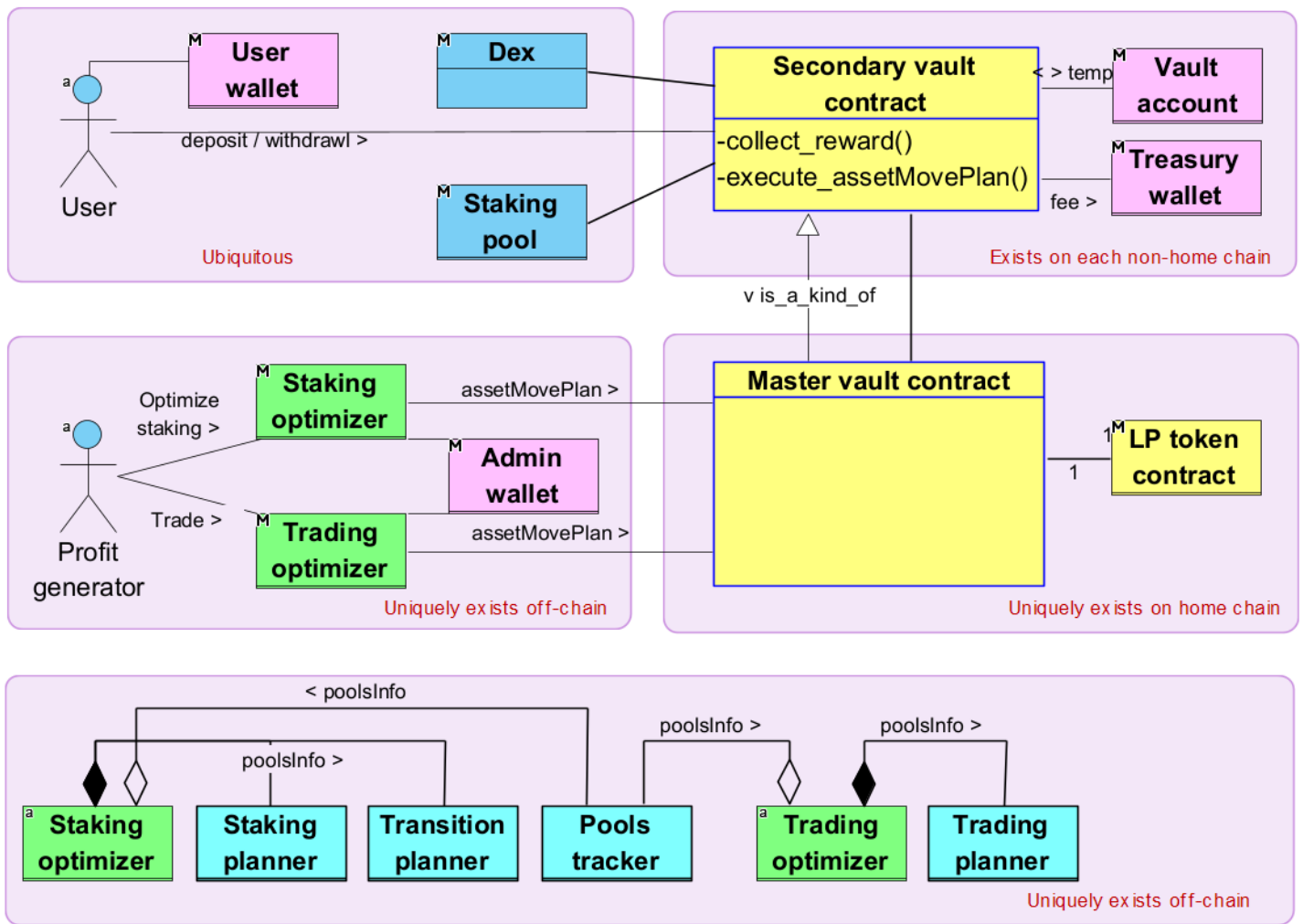


Figure. High-level functional modules of Mozaic DeFi ver. 1.0
Goal: Minimize the functionality of vaults.

Functional modules are described below:

- **Secondary vault contract:** This module is a smart contract and deployed on each chain except the home chain. It participates in the cooperation with its peer **Secondary vault contracts** to implement **Control asset move**. This module has at least the following private operations that work locally on its chain: **collect_reward(...)** and **move_staking_asset(...)**.
- **Master vault contract:** This module is a smart contract, has global operations, in addition to the local operations inherited from **Secondary vault contract**, and is deployed on one and only one of the chains, called **Home chain**, where it takes the role of **Secondary vault contract**, as well as the unique role of the master vault operating **LP token contract**.
- **LP token contract:** This module manages the LP token balances of **Users**, which represents their proportional share of the total assets of the system.
- **User wallet:** This is a blockchain wallet and identifies a **User**. **User's** actions; like deposit, withdraw, and harvest; are authenticated/authorized with this wallet.
- **Vault account:** It is the blockchain account of, and controlled by, **Secondary vault contract** and used to store temporary assets, like funds pending staking.

- **Treasury wallet:** This is a blockchain wallet, and a place to store and retrieve system revenues, like fees. It will be better if it is not owned by a human, but be the account of a smart contract that only obeys vault contracts, for better decentralization. It is deployed on all chains.
- **Staking optimizer:** This is an off-chain module that can invoke **Master vault contract**. This module is globally unique, calculates optimal **assetMovePlans**, and lets the master vault to execute the plans (in cooperation with secondary vaults). *It is an important design decision that the assetMovePlan is calculated off-chain, thus leading to transparency and security debates, for the sake of gas- and time- savings:*
 - Transparency debate: **Users** will not be able to track why the system chose particular **assetMovePlans** technically.
 - Security debate: If the calculation of **assetMovePlan** is hacked or compromised, then the system will make a less-optimal staking maneuver.
 - Justification: Only the second of the following concerns becomes less transparent, leading to both un-assured best profitability and assured huge gas- and time- savings.
 - how much of what assets from which pool to which pool, is the move about
 - whether all the asset moves are securely and/or reasonably/optimally chosen
 - whether all the asset moves are securely executed and logged
 - whether the move logs are readily available to check later
 - whether the execution of assetMovePlan is integrated
- **Trading optimizer:** This off-chain module is similar to **Staking optimizer**, except it relates to trading.
- **Adimin wallet:** This wallet is used to invoke **"Master vault contract"**, in privilege, on behalf of the administrator.
- **Staking planner:** An integral component of **Staking optimizer**, this module predicts the next most profitable **staking_portfolio**, based on **poolsInfo** provided by **Pools tracker**. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fee and effectively disable the system.
- **Transition planner:** An integral component of **Staking optimizer**, this module predicts the most efficient **assetMovePlan**, which is the best procedure of asset move that implements the transitioning to a given **staking_portfolio**, based on the current **poolsInfo**.
- **Trading optimizer:** This is similar to **Staking optimizer**, except that it relates to trading.
- **Trading planner:** This is similar to **Staking planner**, except that it relates to trading.
- **Pools tracker:** A shared module between **Staking optimizer** and **Trading optimizer**, this module retrieves and tracks all relevant information from chains, like Reward Release Speed, and total Staked LP of each pool. Running this module on-chain would enhance transparency, but would at the same time incur huge gas fee and effectively disable the system.

3. Exploring vaults

We have identified vaults through their surrounding modules interacting with them.

The external actors in the following use case diagram, together with their interactions with vaults are already described above.

We can now explore the use cases of vault.

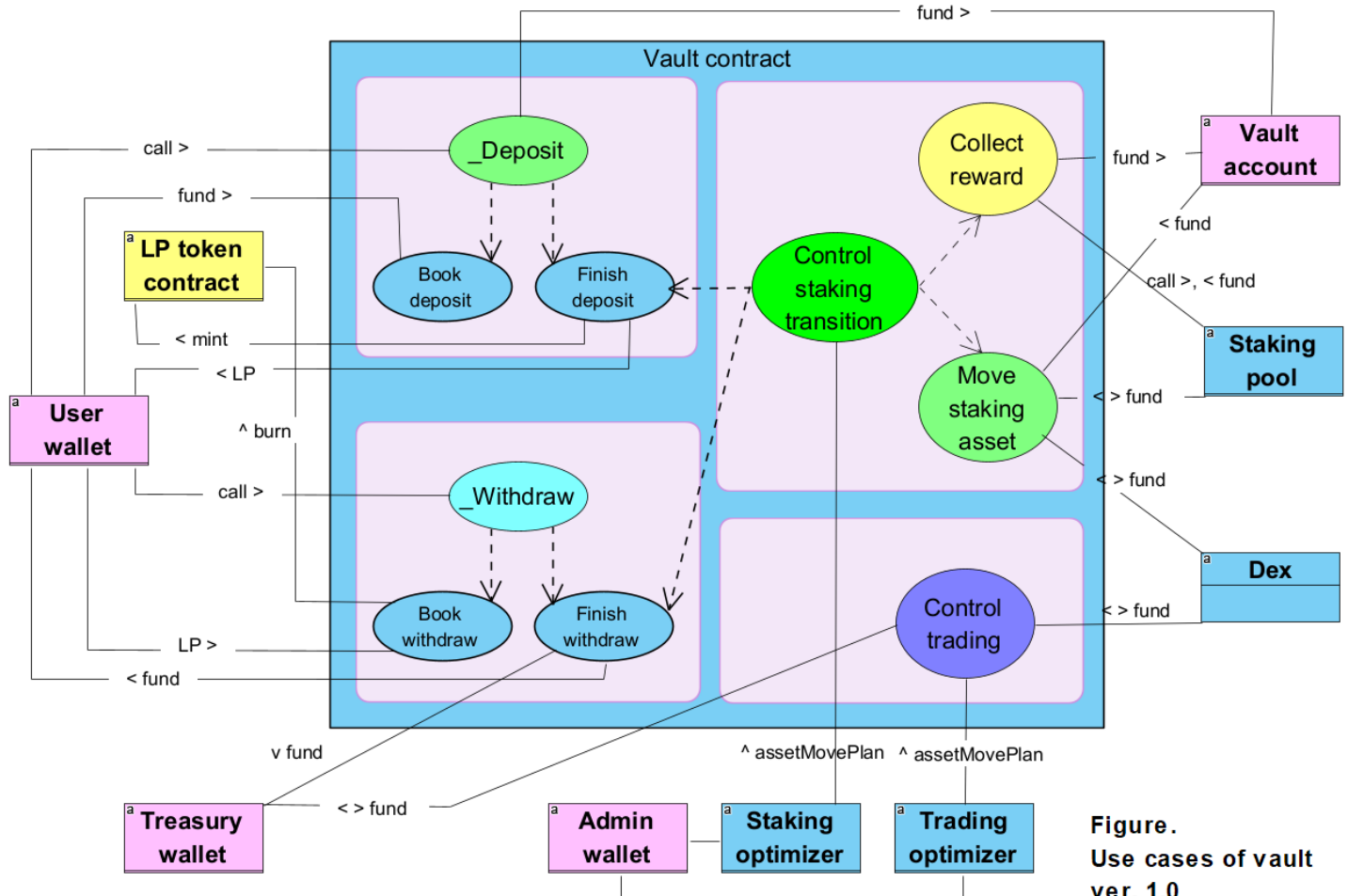


Figure.
Use cases of vault
ver. 1.0

- **_Deposit:** This is what happens at the level of vault contracts when the **Deposit** use case is invoked at the system level. Invoked by the **User wallet** with fund amount to deposit, this use case coordinates the following two use cases.
- **Book deposit:** This use case
 - collects the fund from **User wallet**,
 - books the deposit request with the system,
 - and pauses the session of "_Deposit".
- **Finish deposit:** This use case
 - resume the session of "_Deposit",
 - retrieves the booked deposit request,
 - mints LP tokens to cover the new fund,
 - returns the LP tokens to **User wallet**,
 - and helps **Control staking transition** stake the fund.
- **_Withdraw:** This is what happens at the level of vault contracts when the **Withdraw** use case is invoked at the system level. Invoked by **User wallet** with LP tokens returned, this used case coordinates the following two use cases.
- **Book withdraw:** This use case

- collects the returned LP tokens,
- burns the collected LP tokens,
- books the withdrawal request with the system,
- and pauses the session of "_Deposit".
- **Finish deposit:** This use case
 - resume the session of "_Deposit",
 - retrieves the books withdrawal request,
 - subtract fund, as much as covered by the returned LP tokens, from the total system assets, and
 - returns the fund to **User wallet**
- **Control staking transition:** This use case transitions to a new asset state by executing **assetMovePlan** provided by **Staking optimizer**. (This is the most challenging part of vault implementation.) It does *collectively*, by using **Move staking asset**,
 - **Collect reward**,
 - Collect staked assets, to cover the fund to **Finish withdraw**,
 - **Finish deposit**,
 - **Finish withdraw**,
 - execute remaining part of **assetMovePlan**
- **Control trading:** This use case executes **assetMovePlan** provided by **Trading optimizer**.

4. Algorithm of Staking planner

Note. All errors, like numerical processing rounding and price slippage, are ignored at this stage of architectural design.

4.1 Task definition

- Goal: Calculate best staking portfolio, in order to
 - Save vault contracts long calculations of staking optimization, thus to save gas.
 - Keep vault contract insulated from future algorithm upgrades of staking optimization.
- Consideration
 - Input may not be idealistically consistent inside itself, because an idealistic snapshot of multiple chain states is impossible logically.
 - Output staking portfolio may not be completely/perfectly implemented, because input may have errors and there are unpredictable price slippages sneaked into the calculation.
- Input
 - Deposit requests currently booked
 - Withdrawal requests currently booked
 - Current pending rewards
 - Current poolsInfo
- Process
(See below)
- Output
 - optimal staking portfolio

4.2 Definition

- **Pools state**

$$PS = \{PS_i | i = 1..N\},$$

where PS_i is the i -th pool state:

$$PS_i = (RewardRate_i, RewardToken_i, TotalStake_i, StakingToken_i, MozaicStake_i)$$

Note: We assume reward on PS_i is calculated as:

$$MozaicReward_i = \frac{RewardRate_i}{TotalStake_i} \times MozaicStake_i$$

- **Token vectors**

- **User tokens vector**

$$UserTokens = \{UserToken_i | i = 1..M\}$$

Example: $UserTokens = \{USDT, USDC, ETH, BNB, BTC\}$

- **Reward tokens vector**

$$RewardTokens = \{RewardToken_i | i = 1..N\}$$

- **Staking tokens vector**

$$StakingTokens = \{StakingToken_i | i = 1..N\}$$

- **Tokens vector**

$$Tokens = (UserTokens, RewardTokens, StakingTokens)$$

- **Asset vectors**

- **Vector of booked deposit requests**, at time index t

$$Deposits^t = \{D_i^t | D_i^t : \text{deposit amount, at time } t, \text{ denominated by } UserToken_i, i = 1..M\}$$

Example: $Deposits^t$ of {1, 2, 3} means {1 USDT, 2 USDC, 3 ETH}, assuming $UserTokens = \{USDT, USDC, ETH\}$

- **Vector of booked withdrawals requests**, at time index t

$$Withdrawals^t = \{W_i^t | D_i^t : \text{withdrawal amount, at time } t, \text{ denominated by } UserToken_i, i = 1..M\}$$

- **Vector of collected rewards**, at time index t

$$Rewards^t = \{R_i^t | D_i^t : \text{reward amount, at time } t, \text{ denominated by } RewardToken_i, i = 1..M\}$$

- **Vector of staked assets**, at time index t

$$Stakes^t = \{S_i^t | D_i^t : \text{stake amount, at time } t, \text{ denominated by } StakingToken_i, i = 1..M\}$$

- **Transformations**

- **Transformation $USDT^{+1}$**

$USDT^{+1}$ transforms a Tokens-denominated asset vector to USDT-denominated vector, with market exchange rates.

Example: $USDT^{+1}$ transforms (1, 2, 3) to (1, 2.02, 1300), assuming $UserTokens = (USDT, USDC, ETH)$ and $USDT/USDC = 1.01$ and $USDT/ETH = 1300$.

- **Transformation $USDT^{-1}$**

$USDT^{-1}$ transforms a USDT-denominated asset vector to Tokens-denominated vector, with market exchange rates.

Example: $USDT^{-1}$ transforms (1, 2.02, 1300) to (1, 2, 3), assuming $UserTokens = (USDT, USDC, ETH)$ and $USDT/USDC = 1.01$ and $USDT/ETH = 1300$.

- **Transformation FOP**

FOP , standing for Find Optimal Portfolio, finds the best USDT-denominated vector of staked assets, for a given total USDT amount.

Example: FOP transforms (12345) to the best staking of 123 USDT over staking pools, and has the format of USDT-denominated $Stakes^t$, like (100, 20, 3) all in USDT, assuming we have a total 3 pools.

- **Transformation Sum**

$ElementWiseSum$ sums up all elements of a vector.

- **Mozaic's asset state**, at transition t

- Asset snapshot just before the transition that takes place at time t :

$$MS^t = (Tokens, Deposits^t, -Withdrawals^t, Rewards^t, Stakes^t)$$

$$\text{Or, simply, } MS^t = (T, D^t, -W^t, R^t, S^t)$$

- Asset snapshot just after the transition that takes place at time t :

$$MS^{t+} = (Tokens, 0Deposits, -0Withdrawals, 0Rewards, optimal\ Stakes^t)$$

$$\text{Or, simply, } MS^{t+} = (T, 0D, -0W, 0R, optimal\ S^t)$$

, where 0D, 0D, and 0R are a vector of zero values in their respective vector lengths.

4.3 Formulae

If a state transition takes place at time t , Mozaic's asset state MS^t changes to MS^{t+} as shown in the following diagram:

$$\begin{array}{ccc}
MS^t = (T, D^t, -W^t, R^t, S^t) & \xrightarrow{(Resulting\ transition)} & MS^{t+} = (T, 0D, -0W, 0R, optimal\ S^t) \\
\downarrow USDT^{+1} & & \uparrow USDT^{-1} \\
MS_U^t = (T, USDT^{+1}(D^t), \dots) & \xrightarrow{(Implicit)} & MS_U^{t+} = (T, 0D, -0W, 0R, FOP(Total\ in\ USDT)) \\
\downarrow Sum & & \uparrow zeros, FOP \\
Total\ in\ USDT & \xrightarrow{Identity} & Total\ in\ USDT
\end{array}$$

The algorithm for Staking planner MS^t is a chain of transformations:

$$optimal\ S^t = USDT^{-1} \circ FOP \circ Sum \circ USDT^{+1}(T, D^t, -W^t, R^t, S^t)$$

- $USDT^{+1}$ and $USDT^{-1}$ are obvious, except that we may need systematic methods to find best Dexes and swap paths.
- FOP is solved analytically, demonstrating about 9% of competitive edge over the public.
- ElementWiseSum is trivial.
- D^t and W^t can be retrieved from the booked requests of deposits and withdrawals.
- S^t is found when we "Collect reward" pending rewards.

The formula essentially does:

- collect all available assets: deposits pending staking, less withdrawals pending, plus pending rewards, and the current staking.
- transform them to USDT and sum up to get a single number: Total in USDT.
- allocate the Total in USDT optimally across all staking pools.
- transform the allocated USDT amount back to the native tokens for the staking pools.

5. Algorithm for Transition planner

5.1 Mozaic LP token will only be present on the home chain

- LP token can be deployed on any chain(s). According to the requirements, users can deposit any token on any chain and withdraw any token on any chain. It doesn't matter which chain their LP token belongs to.
- If LP token is deployed on multiple chains, there might be significantly bad user experience and cost overhead, because collecting assets from a wallet requires the user's engagement and wallet connection.
 - A user's wallet may have multiple versions of LP token, although they equally represent the user's, global, Stock share. This poses inconvenience to withdrawal UI and to users. Users will have to think in terms of "how much of what LP tokens to return", instead of "how much LP tokens to return".
 - Suppose the user chooses multiple LP versions to return. Either off-chain modules will have to ask the user to switch the wallet to each of the multiple chains, in turn, to enable the vault on that chain to collect back the LP token on the chain, in an inconvenient and in-secure way; or the user will have to invoke withdrawal process multiple times each focusing on one of the LP versions; in an inconvenient and secure way. Or, the user will have to swap the LP versions into a single LP version, before invoking withdrawal.
 - When depositing, on the other hand, users will have to choose which LP token version to receive, although they equally represent the user's global Stock share. The vault that is serving a user's deposit, will have to send an LP message to the chain where the user-chosen LP version presents, if the user's deposit token and their chosen LP version are on different chains. (Batch messaging might mitigate this a little.)
- If LP token is deployed on multiple chains, we will have to allocate huge initial liquidity on those LP liquidity pairs, for little convenience for traders when trading is not yet important.
- If there is critics that solo LP version, which is naturally deployed on the home chain, will force Mozaic system to transfer to, or send LZ message to, the, non-home, chain on which the user wants to withdraw a token type, then we can say the message will rarely serve a single user but mostly several users in batch, because the withdrawals happen while transitioning to a new staking when all booked deposit requests are handled.

5.2 Task definition for Transition planner

- Goal: Calculate best transition asset flow plan, in order to
 - Save vault contracts long calculations of staking optimization, thus to save gas.
 - Keep vault contracts insulated from future algorithm upgrades of staking optimization.
- Consideration
 - Input may not be idealistically consistent inside itself, because an idealistic snapshot of multiple chain states is impossible logically.
 - Output may not be completely/perfectly implemented, because input may have errors and there are unpredictable price slippages on Dexes.
- Input (see section 4. Algorithm of Staking planner, for their definitions)
 - $Deposits^t$
 - $Withdrawals^t$
 - $Rewards^t$
 - $Stakes^t$

- $poolsInfo^t$
- $optimal\ Stakes^{t+}$, or $optimalS^{t+}$
- Process
 - Find a step-by-step logical procedure of asset flow instructions that implements the following transition:

$$MS^t = (T, D^t, -W^t, R^t, S^t) \xrightarrow{(Resulting\ transition)} MS^{t+} = (T, 0D, -0W, 0R, optimal\ S^t)$$
 - Regularize the procedure by Removing mutually/transitively cancelling asset moves in the procedure
 -
- Output
 - $assetMovePlan^t$

5.3 Formulae (under construction)

- Both deposit and withdrawal have future tense.
 - when a user deposits, let them know the approximate LP amount based on the past sync.
 - when a user withdraws, let them know the approximate token amount based on the past sync.
 - on-chain
 - on-chain: M: Snapshot withdraw requests. send a withdrawal requests.
 - on-chain: S: Snapshot local S and deposit requests. Collect R.
 - on-chain: S->M send local S and local R
 - on-chain: Find USDT(S+R)
 - on-chain: Find W based on their LP, Total LP, and USDT(S+R). Do not return W yet.
 - on-chain: Find LP for D, based on USDT(D), Total LP, and USDT(S+R). Return LP at the same time.
 - off-chain: Find assetMovePlan, based on S, R, D, W, and poolsInfo.
 - on-chain: Return W after transitioning to the optimal S.
 - on-chain: Execute assetMovePlan, finishing D and W, furnished by assetMovePlan.
- rebalance fee will be imposed on withdrawal if it makes more deficit of a token in the coming opt.
- rebalance fee will be imposed on deposit if it makes more surplus of a token in the coming opt.